



University of Southern Brittany – UBS

Course Name:

M1-ISC-CSSE-CYBERUS - Applied Cryptography

\$

Lecturer: Professor Guy GOGNIAT

LAB REPORT – 3 Part 1

(Asymmetric cryptography algorithms)

Submitted by :

Name; Assan Jallow

Student ID: E2506147

Email: jallow.e2506147@etud.univ-ubs.fr

Lab 3: Part-1 Asymmetric cryptography algorithms

Question 1:

How works the algorithm? Explain the code.

Answer:

`rabinMiller(num)` is a probabilistic primality test. It writes $\text{num}-1 = 2^t \cdot s$ with s odd, then does several trials:

1. Pick random base a in $[2, \text{num}-2]$.
2. Compute $v = a^s \bmod \text{num}$. If $v == 1$ or $v == \text{num}-1$, this trial “passes”.
3. Otherwise square v repeatedly up to $t-1$ times: $v = v^2 \bmod \text{num}$. If you ever get $\text{num}-1$, the trial passes; if you finish without hitting $\text{num}-1$, num is composite.
4. If all (e.g., 5) trials pass, num is probably prime. If any trial fails, it's composite. The code implements exactly this loop with counters s , t , and the inner squaring.

Question 2:

Explain the algorithm.

Answer:

`generateLargePrime(keysize)` finds an RSA-style prime by repeatedly picking a random integer with exactly `keysize` bits (i.e., in the range $[2^{k-1}, 2^k)$) and testing it for primality. The test `isPrime(num)` first does a fast screen against a small list of low primes (`LOW_PRIMES`) to eliminate easy composites, and only if the number passes that screen does it run the stronger probabilistic Rabin–Miller test (`rabinMiller(num)`). Rabin–Miller gives high confidence that a number is prime by trying to “witness” compositeness with random bases; if none expose a factor after several trials, the number is accepted as probably prime. The loop repeats until the first candidate passes these checks, at which point the function returns that prime and reports how many attempts were needed.

Question 3:

Complete the `generateKey` function and explain your solution.

Answer:

The `generateKey` function creates an RSA public and private key pair. First, it generates two large and different prime numbers, p and q , each roughly half the size of the total key, and multiplies them to form n , which is part of both keys. It then calculates Euler’s totient function, $\phi = (p -$

$1)(q - 1)$, which is needed to compute the private key. Next, it selects a public exponent e , usually 65537 because it is a commonly used, secure, and efficient value; however, if e is not coprime with ϕ , it randomly selects another valid odd number. After that, it computes the private exponent d , which is the modular inverse of e modulo ϕ , meaning d satisfies $e \times d \equiv 1 \pmod{\phi}$. Finally, it returns the public key (n, e) and private key (n, d) . This process ensures that the keys are mathematically linked for encryption and decryption but secure enough that recovering the private key from the public key is computationally hard.

Question 4:

Explain the makeKeyFiles function

Answer:

The makeKeyFiles function is responsible for generating RSA key pairs and saving them into files. First, it checks whether key files with the same name already exist to avoid overwriting them by mistake. If the files do not exist, the function calls generateKey(keySize) to create a public and private key. After generating the keys, it saves the public key in a file named name_pubkey.txt, which contains the key size, n , and e , separated by commas. Similarly, it saves the private key in name_privkey.txt, which contains the key size, n , and d . The function also prints useful information, such as the number of digits in the generated keys, to show that the keys were successfully created. The values are stored in a simple comma-separated format so they can be easily read by other functions later. This makes the keys reusable for encryption and decryption tasks.

Question 5:

Complete the code and explain the encryption process.

Answer:

RSA encrypts each plaintext block M as $C = M^e \bmod n$ using the public key (n, e) . The helper getBlocksFromText maps ASCII bytes to big integers (base 256). The block size (in bytes) must be $\leq \text{key size (in bits)}/8$ so that each $M < n$.

Question 6:

Instead of writing the message, propose a new function that read a file containing the plaintext (message) and save it in the message variable.

Code: def read_plaintext_from_file(path, encoding='utf-8'):
--

```
with open(path, 'r', encoding=encoding) as f:
    return f.read()

# Usage in test cell:
message = read_plaintext_from_file('my_message.txt')
```

Question 7:

Complete the code and explain the decryption process.

Answer:

RSA decryption computes $M = C^d \bmod n$ using the private key (n, d) . After turning each ciphertext block back into its plaintext integer, `getTextFromBlocks` reconstructs the original string (tracking `messageLength` to trim any padding space in the last block).

Question 8:

Instead of printing the message, propose a new function that writes a file containing the plaintext.

Answer:

```
Code;
def write_plaintext_to_file(path, text, encoding='utf-8'):
    with open(path, 'w', encoding=encoding) as f:
        f.write(text)

# Example usage after decryption:
# decryptedText = readFromFileAndDecrypt(filename, privKeyFilename)
# write_plaintext_to_file('decrypted_output.txt', decryptedText)
```

Question 9:

Run the cells below and explain how the Digital Signature Algorithm works. Modify the message or the signature and check if the algorithm detects it.

Answer:

In this notebook, ECC on curve P-256 with DSS (FIPS-186-3) and SHA-256 is used to create and verify digital signatures: a private/public key pair is generated with `ECC.generate('P-256')`, the message is hashed with `SHA256.new(my_message)`, and a signature is produced by `DSS.new(privkey, 'fips-186-3').sign(h)` (ECDSA internally uses a fresh random nonce k never reuse it). To verify, the receiver recomputes the hash of the received message and runs `verifier.verify(h_received, received_signature)` with the public key; if the signature corresponds to the exact message and key, it reports that the message is authentic, otherwise it raises an error and indicates it is not authentic. If you modify even a single byte of the message (e.g., change #4355 to #4356) or flip any byte of the signature, verification fails, because the mathematical relation linking the signature pair (r, s) , the hash of the precise message, and the public key no longer holds.

